



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO

Instituto Tecnológico de San Juan del Río



Tópicos de programación

Decompilación con Ghidra C a Python

P R E S E N T A:

DAVID ALY GONZALEZ PEREZ 22590328
MARIA ALONDRA ORTIZ BENÍTEZ 2259034
JESUS ÁVILA CHAPARRO 22590317

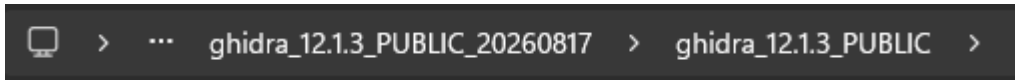
Ingeniería en Sistemas Computacionales

PERIODO Agosto – Diciembre 2026

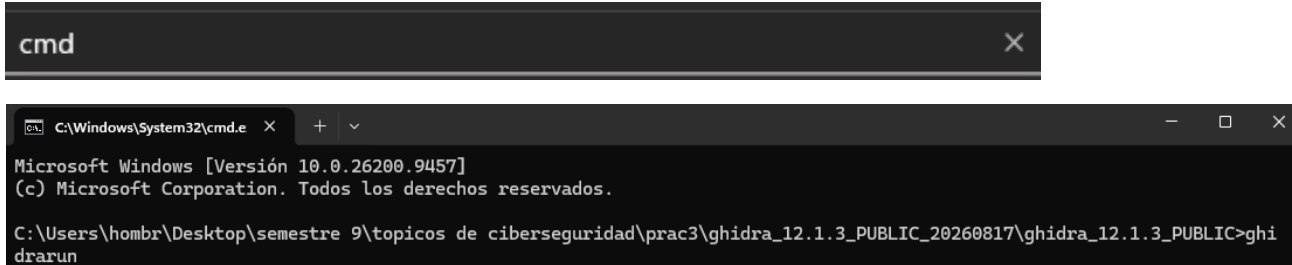




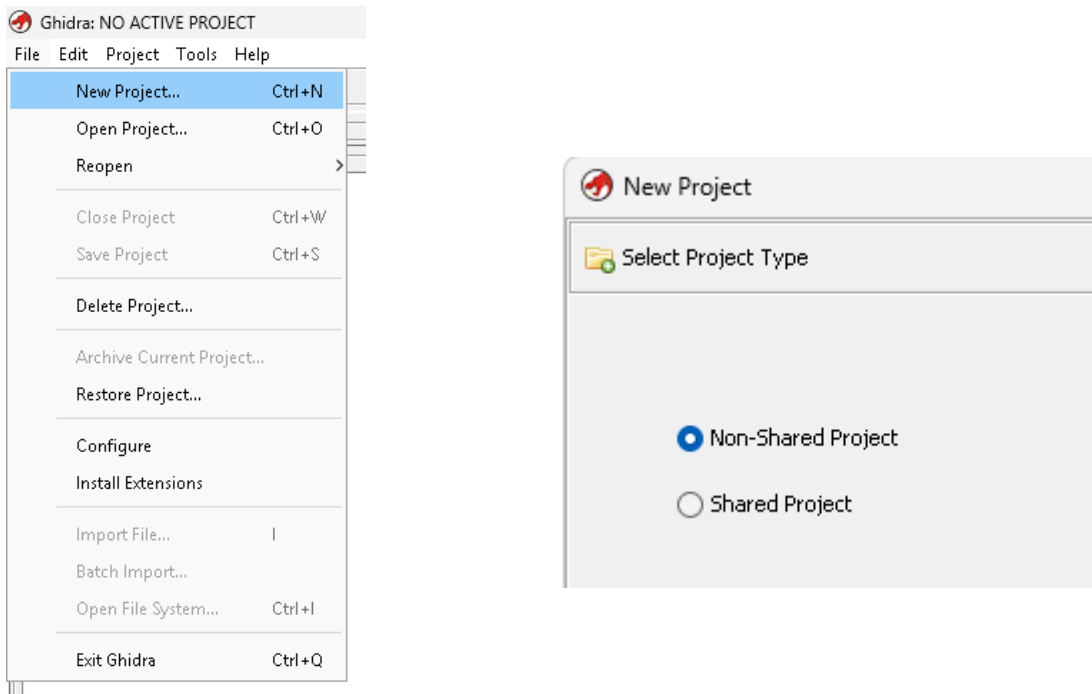
Se busca la ubicación del programa de GhidraRun



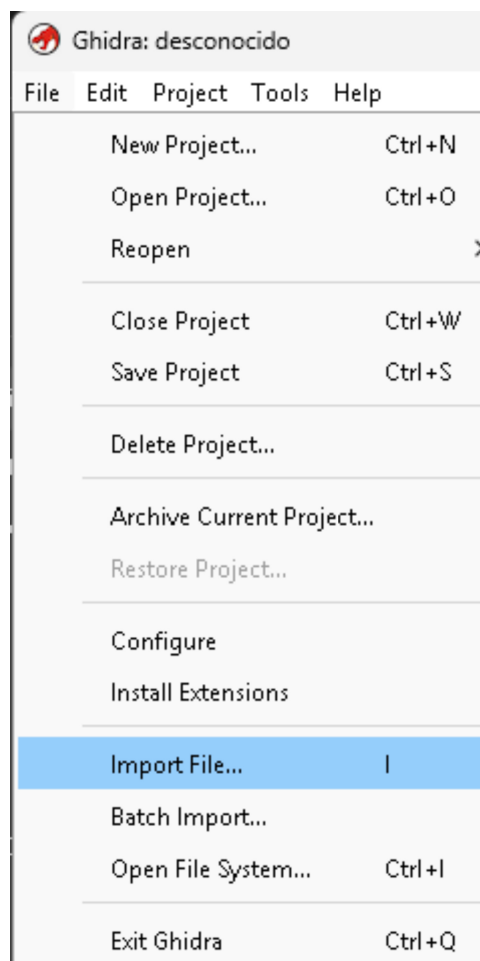
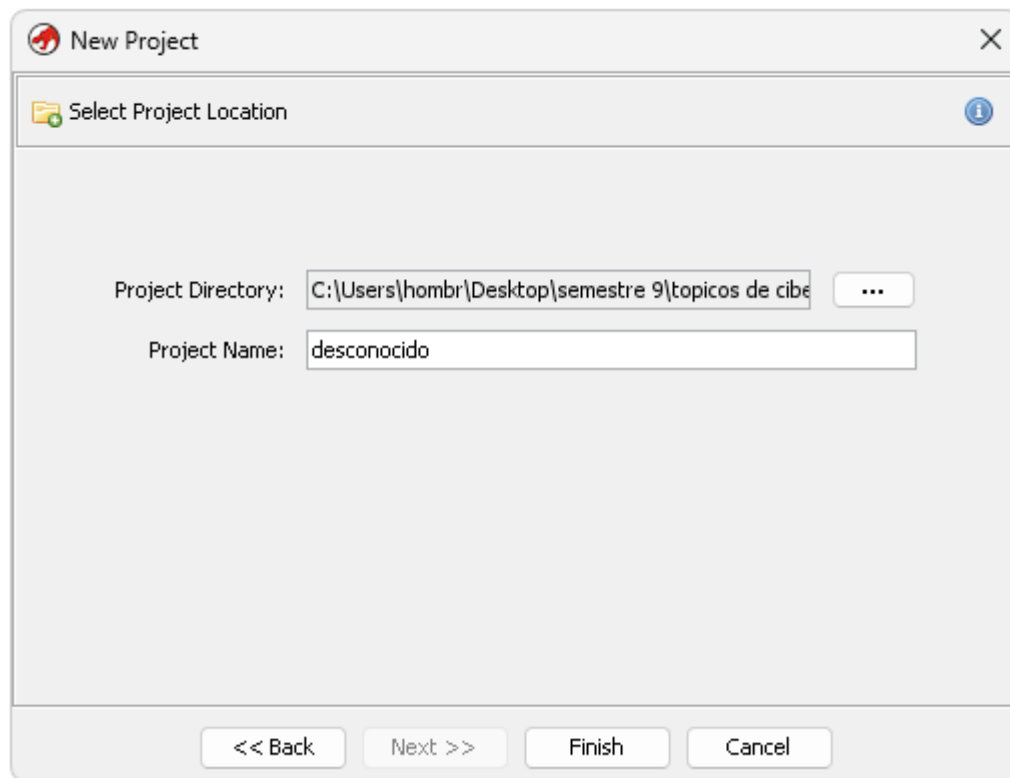
Se ejecuta el programa por medio del CMD con la sentencia ghidrarun



Se crea un nuevo proyecto y se selecciona como un proyecto no compartido



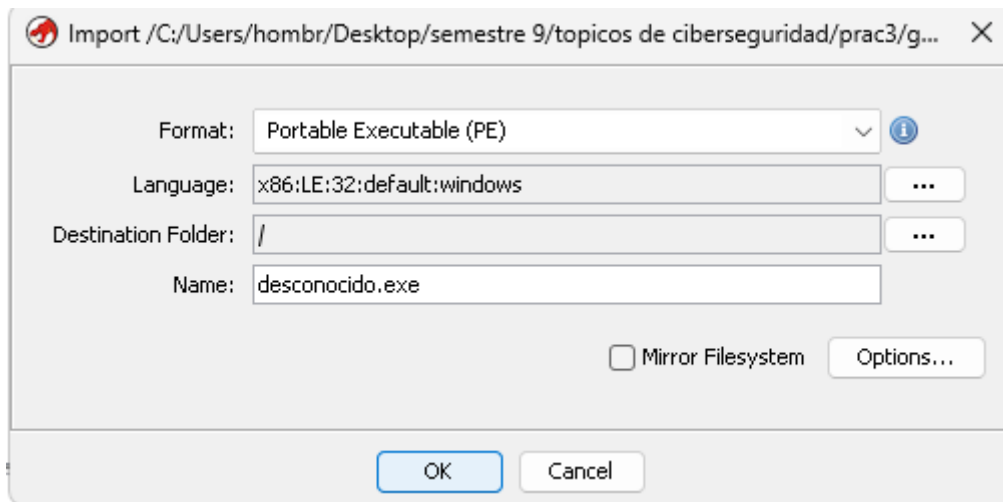
Se elige la carpeta donde se guardara el proyecto



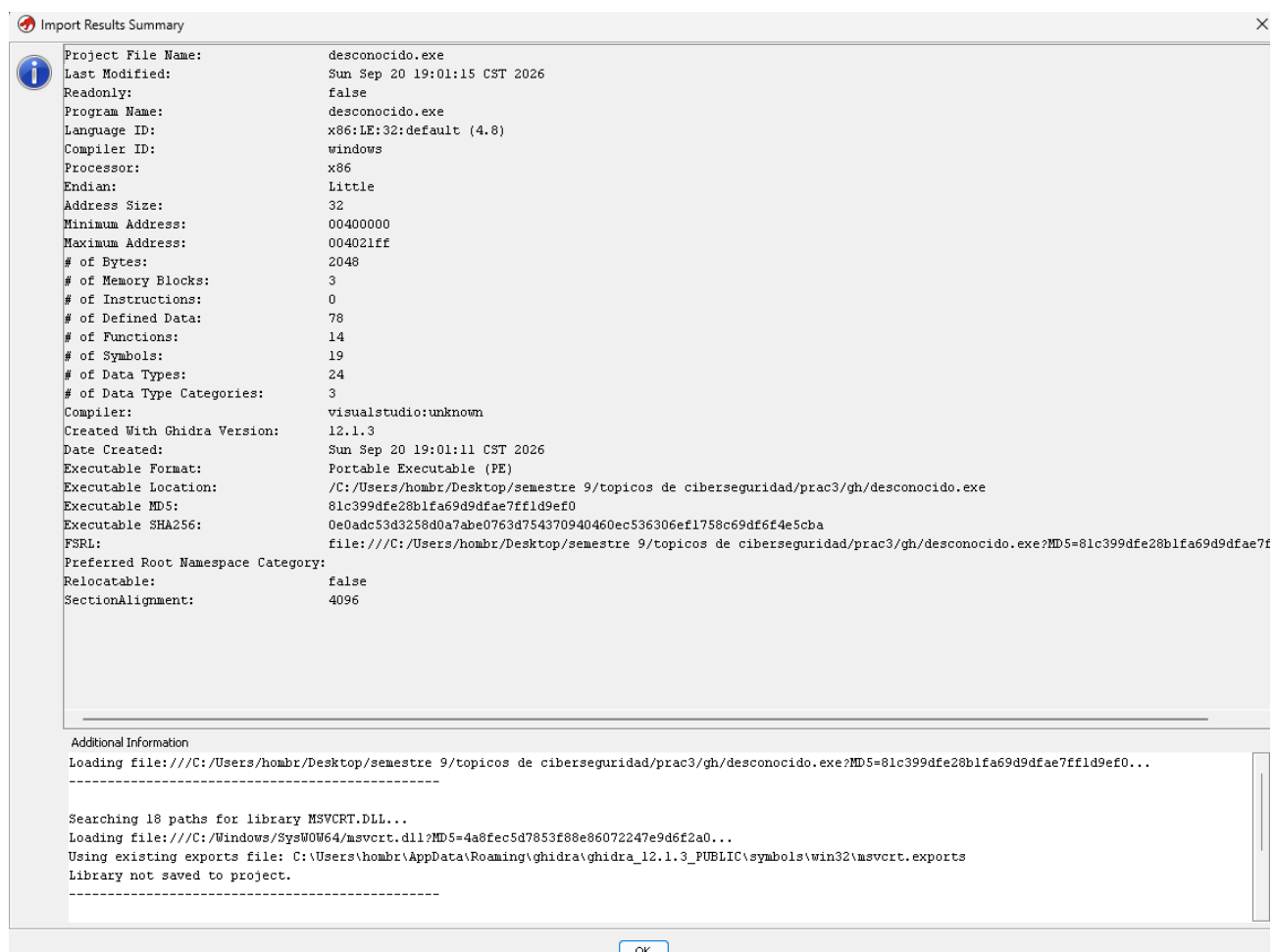
Se importa el archivo “.exe” que se desea analizar

Una vez seleccionado se trabaja con los ajustes por defecto

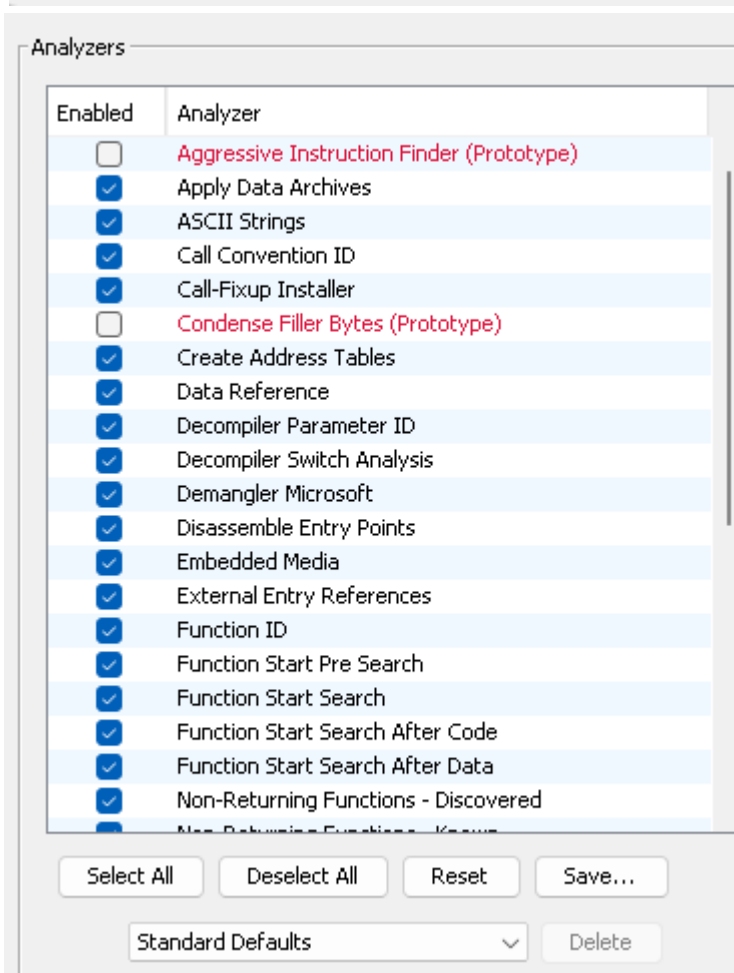
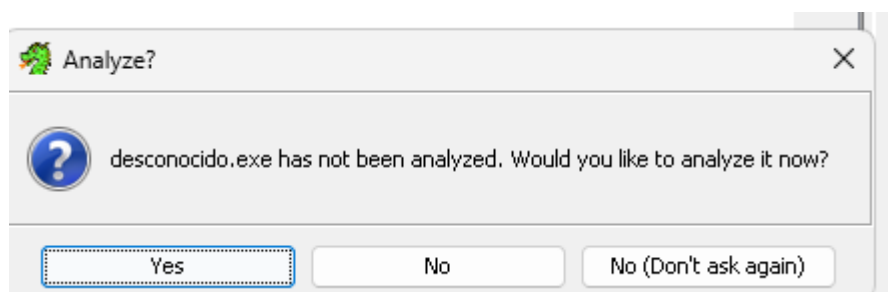
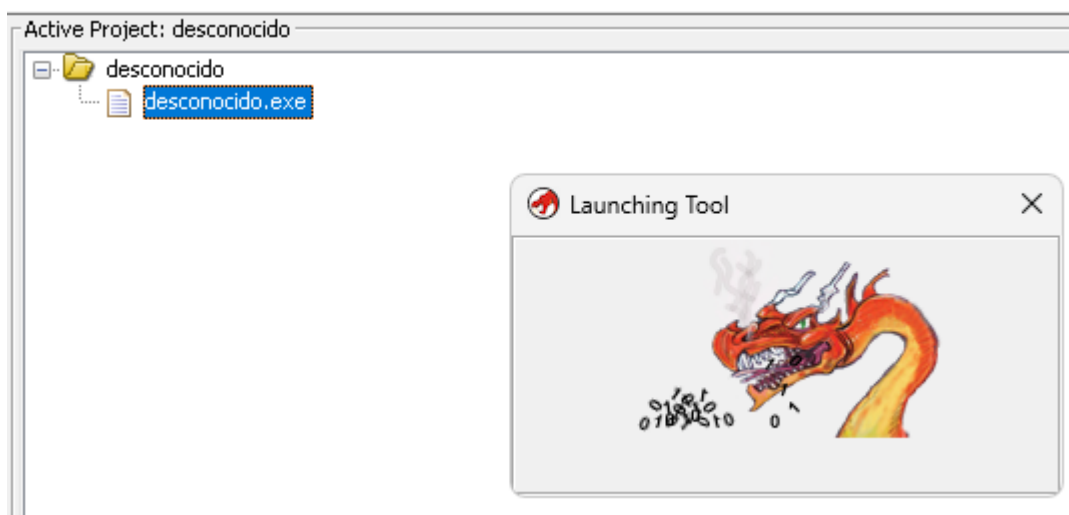
Se despliega la información



correspondiente al archivo:



Se selecciona el archivo a analizar



Se dejan por defecto los analizadores

Al abrirse el archivo es posible observar el __getmainargs el cual trabaja como main, despues de este se encuentra la funcion principal con la cual trabajemos

```

Listing: desconocido.exe
004011ba 20 40 00 MOV ECX,dword ptr [->MSVCRT.DLL: __argv]
004011c0 20 40 00 MOV EDX,dword ptr [->MSVCRT.DLL: __environ]
004011c6 20 40 00 MOV dword ptr [EBP + local_24],EAX
004011c9 8d 45 e4 LEA EAX,<=>local_20,[EBP + -0x1c]
004011cc 50 PUSH EAX
004011cd 8b 05 d8 MOV EAX,dword ptr [DAT_004021d8]
004011d3 50 PUSH EAX
004011d4 52 PUSH EDX
004011d5 51 PUSH ECX
004011d6 8b 45 e0 MOV EAX,dword ptr [EBP + local_24]
004011d9 50 PUSH EAX
004011da e8 91 01 CALL MSVCRT.DLL: __getmainargs
004011df 83 c4 14 ADD ESP,0x14
004011e2 8b 05 ac MOV EAX,dword ptr [->MSVCRT.DLL: __argv]
004011e8 8b 0d b0 MOV ECX,dword ptr [->MSVCRT.DLL: __argv]
004011ee 8b 15 b4 MOV EDX,dword ptr [->MSVCRT.DLL: __environ]

Decompile: entry - (desconocido.exe)
1 void entry(void)
2 {
3     int _Code;
4     _startupinfo local_20;
5     int local_1c [6];
6     FUN_004012e9(local_1c);
7     local_20.newmode = 0;
8     __set_app_type(1);
9     __controlfp(0x10000,0x30000);
10    __getmainargs((int *)__argv_exref,(char ** __argv_exref,(char ** __envir
11    <local_20);
12    _Code = FUN_00401000(*(int *)__argv_exref,*(int *)__argv_exref);
13    /* WARNING: Subroutine does not return */
14    exit(_Code);
15 }

```

Se selecciona con doble click la función para poder abrirlo y observar como es que trabaja la función para poder traducirlo despues a python.

```

00401202 50 PUSH EAX
00401203 e8 f8 fd CALL FUN_00401000
15 _Code = FUN_00401000(*(int *)__argv_exref,*(int *)__argv_exref);
16 /* WARNING: Subroutine does not return */

```

```

Stack[-0xd]:1 local_d XREF[1]
Stack[-0x14]:4 local_14 XREF[9]

FUN_00401000 XREF[4]: 00401000, 00401001, 00401003, 00401009, 0040100a, 0040100d, 00401010, 00401016, 0040101b, 0040101c, 00401021

00401000 55 PUSH EBP
00401001 89 e5 MOV EBP,ESP
00401003 81 ec 10 SUB ESP,0x10
00401009 90 NOP
0040100a 8b 45 08 MOV EAX,dword ptr [EBP + param_1]
0040100d 83 f8 01 CMP EAX,0x1
00401010 0f 8f 18 JG LAB_0040102e
00401016 b8 00 20 MOV EAX,s_Usage:_sumnottransfer_archivont_00402000
0040101b 50 PUSH EAX,<=>s_Usage:_sumnottransfer_archivont_00402000
0040101c e8 07 03 CALL MSVCRT.DLL: printf
00401021 83 c4 04 ADD ESP,0x4

Decompile: FUN_00401000 (desconocido.exe)
1 __cdecl FUN_00401000(int param_1,int param_2)
2 {
3     undefined4 uVar1;
4     FILE *_File;
5     int iVar2;
6     int iVar3;
7     time_t tVar4;
8     if (param_1 < 2) {
9         printf(s_Usage:_sumnottransfer_archivont_00402000);
10        uVar1 = 1;
11    }
12    else {
13        tVar4 = time((time_t *)0x0);
14        srand((uint)tVar4);
15        _File = fopen(*(char **)(param_2 + 4),<DAT_0040201f);
16        if (_File == (FILE *)0x0) {
17            printf(s_No_se_puede_abrir_el_archivont_00402022,0);
18            uVar1 = 1;
19        }
20        else {
21            do {
22                iVar2 = fgetc(_File);
23                if (iVar2 != -1) {
24                    iVar3 = rand();
25                    iVar3 = iVar3 % 0x100;
26                    if (iVar2 < iVar3) {
27                        printf(s_%d-%d_00402042,iVar3,iVar3 - iVar2);
28                    }
29                }
30            } while (iVar2 != -1);
31        }
32    }
33    return uVar1;
34 }

```

```
''' #####  
código de C traducido a Python
```

```
undefined4 __cdecl FUN_00401000(int param_1,int param_2)
```

```
{  
    undefined4 uVar1;  
    FILE *_File;  
    int iVar2;  
    int iVar3;  
    time_t tVar4;  
  
    if (param_1 < 2) {  
        printf(s_Usage:_sumnottransfer_archivont_00402000);  
        uVar1 = 1;  
    }  
    else {  
        tVar4 = time((time_t *)0x0);  
        srand((uint)tVar4);  
        _File = fopen(*(char **)(param_2 + 4),&DAT_0040201f);  
        if (_File == (FILE *)0x0) {  
            printf(s_No_se_puede_abrir_el_archivont_00402022,0);  
            uVar1 = 1;  
        }  
        else {  
            do {  
                iVar2 = fgetc(_File);  
                if (iVar2 != -1) {  
                    iVar3 = rand();  
                    iVar3 = iVar3 % 0x100;  
                    if (iVar2 < iVar3) {  
                        printf(s_%d-%d_00402042,iVar3,iVar3 - iVar2);  
                    }  
                    if (iVar3 < iVar2) {  
                        printf(s_%d+%d_00402049,iVar2 - iVar3,iVar3);  
                    }  
                    if (iVar2 == 0) {  
                        printf(&DAT_00402050);  
                    }  
                    if (iVar3 == iVar2) {  
                        printf(s_%d+1_00402055,iVar2 + -1);  
                    }  
                }  
            } while (iVar2 != -1);  
            fclose(_File);  
            uVar1 = 0;  
        }  
    }  
}
```

```
}  
return uVar1;  
}  
'''
```